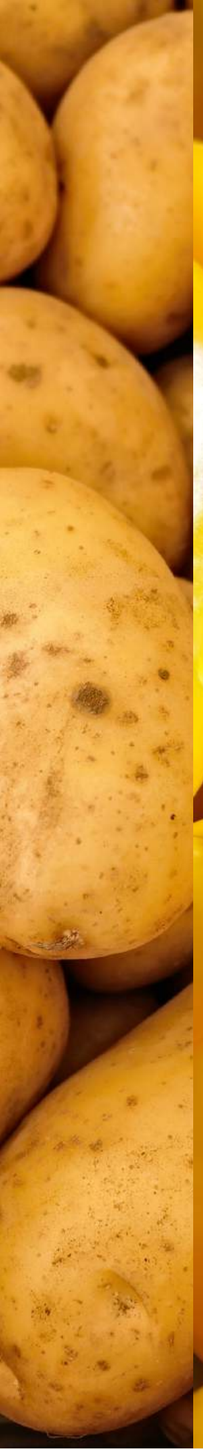
ABC FOODMART

DATABASE DESIGN WITH SQL & PYTHON

Group 7: Lauren Tran, Hitanshi Shah, Milson Chhantyal, Nicholas Scardigno



Business Scenario

- Our team acted as consultants to modernize ABC Foodmart's outdated data systems (spreadsheets + paper)
- Business expanding from 2 stores in Queens to 5 total with 3 new in Brooklyn & current system cannot scale efficiently
- Weaknesses: fragmented, hard to ensure accuracy, infrequent updating
- Addressing customer needs:
  - Centralized system to replace spreadsheets/manual tacking
  - Real-time visibility into sales, inventory, and operations
  - Track performance across multiple locations
  - Impact: transforms data into a strategic asset for growth, improved decision making

Initial Plan

- Planned to create a centralized relational database where all business data would be consolidated and stored
- Database would put all operational data into structured tables with clearly defined relationships
- Focus schema on the following business activities: staffing & scheduling, products & inventory, suppliers & purchasing, sales, financials, and store structure
- Design queries and dashboards that provide real-time insights into key business metrics

Data

- Consists of 20 CSV files, each corresponding to a database table
- Generated with ChatGPT to simulate real grocery store operations
- Workflow
  - Defined SQL schema for each table → provided schemas as input with specific constraints and details → Generated CSV files iteratively
- Example inputs
  - "Create a stores CSV file based on the stores table schema. There should be 5 store locations, with 2 located in Queens and 3 located in Brooklyn"
  - Generate CSV files for the following tables: shifts, employee_schedules, and time_off_requests. Ensure correct foreign key relationships, valid employee IDs, and consistency with the 5 store locations."

```
df = pd.read_csv("products.csv")
```

```
df.head()
```

	product_id	product_name	category_id	brand_id	unit_size
0	1	Bananas	1	9	32 oz
1	2	Apples Gala	1	7	20 oz
2	3	Strawberries	1	7	12 oz
3	4	Lettuce Romaine	1	10	64 oz
4	5	Tomatoes Vine	1	18	2 lb

github link: https://github.com/laurentran2022/APAN5310_Group7

Database Design

- Structured in Third Normal Form (3NF) to reduce redundancy and improve consistency
- 20 tables covering operations: staffing, inventory, suppliers, sales, and financials
- One to Many Relationships (enforced through foriegn keys)
  - stores → employees
  - employees → time_off_requests
- Many to Many Relationship (implemented via junction table with foreign keys)
  - suppliers → supplier_products ← products
- Enforced integrity using:
  - Primary & foreign keys
  - Constraints (CHECK, UNIQUE)
  - 1 Trigger: employees can only be scheduled at assigned store

github link: https://github.com/laurentran2022/APAN5310_Group7

Database Implementation

```
import psycopg2

conn = psycopg2.connect(
    host='localhost',
    port=5432,
    user='postgres',
    password='123',
    dbname='postgres'
)
conn.autocommit = True
cursor = conn.cursor()
```

```
cursor.execute("DROP DATABASE IF EXISTS
foodmart_db")
cursor.execute("CREATE DATABASE
foodmart_db")

cursor.execute("""
CREATE TABLE stores (
  store_id SERIAL PRIMARY KEY,
  store_name VARCHAR(100) NOT NULL,
  borough VARCHAR(50) NOT NULL,
  opening_date DATE
);
""")
```

- Implemented in PostgreSQL with Python (psycopg2)
- SQL executed via cursor.execute()
- Enables fully reproducible & consistent setup
- Structured table creation with enforced relationships, primary keys, foreign keys, trigger, and constraints
- Consistent, testable, and scalable database setup

github link: https://github.com/laurentran2022/APAN5310_Group7

ETL Procedure

- Built ETL pipeline using Python (pandas) + SQLAlchemy
- Extract: CSV data loaded into DataFrames
- Transform: minimal cleaning due to schema alignment
- Load: inserted into PostgreSQL tables in dependency order
- Focus on:
  - Data integrity
  - Structural consistency
  - Efficient loading
- Outcome: reliable pipeline for structured data ingestion

```
from sqlalchemy import create_engine

engine = create_engine(

"postgresql+psycopg2://postgres:123@localhost:5432/foodmart_db"
)

with engine.connect() as conn:
    print("Connected successfully")
```

```
import pandas as pd
stores = pd.read_csv("stores.csv")
```

```
stores_df.to_sql(name='stores', con=engine, if_exists='append', index=False)
print(f"{len(stores_df)} rows inserted into stores.")
```

Analytical Framework

- Integrated data across sales, inventory, staffing, and suppliers
- Generated actionable insights using SQL queries:
  - Revenue by store
  - Sales by category
  - Inventory shortages
  - Supplier delays
  - Etc.
- Enabled identification of:
  - Performance trends
  - Operational inefficiencies
  - Profitability drivers
- Shift from basic reporting → data-driven decision-making

Query | Query History

```sql
SELECT
  s.store_name,
  SUM(st.total_amount) AS revenue
FROM sales_transactions st
JOIN stores s
  ON st.store_id = s.store_id
GROUP BY s.store_name
ORDER BY revenue DESC;
```

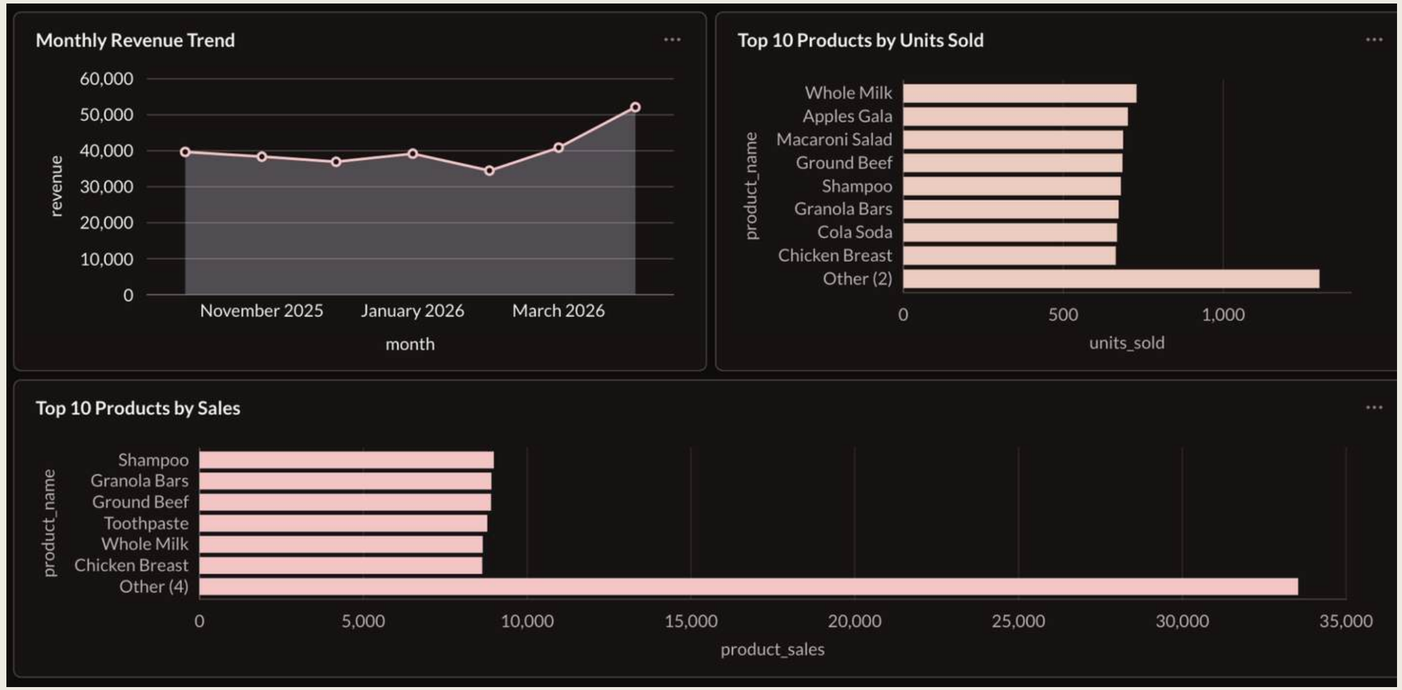
Data Output | Messages | Notifications

	store_name text	revenue double precision
1	FoodMart Brooklyn - Park Slope	57113.7500000000
2	FoodMart Queens - Astoria	54955.5700000000
3	FoodMart Brooklyn - Williamsburg	53624.9300000000
4	FoodMart Brooklyn - Bushwick	53448.6399999999
5	FoodMart Queens - Jackson Heig...	51298.4200000000

Metabase Dashboards

Executive Dashboard Live Demo

- Built interactive dashboards for different business functions
- Dashboards include:
  - Executive Dashboard
  - Finance/Accounting → revenue vs expenses
  - Inventory → stock levels & shortages
  - Purchasing & Vendor → total purchase orders & late deliveries
  - Sales Performance → trends & product performance
  - Staffing → employees by role & time off requests
  - Brooklyn Expansion → projected revenue & supplier capacity



System Implementation & User Interaction

- Analysts interact via SQL queries in PostgreSQL (pgAdmin)
- Executives use Metabase dashboards for non-technical insights
- System bridges technical and non-technical users
- Meets the customer’s needs to get insights into:
  - Automated reporting
  - Real-time analytics
  - Scalable access to insights
- Replaces inefficient manual analysis (Excel-based workflows)

Infrastructure, Performance & Scalability

- Built with performance in mind (indexes, normalized schema)
- Additional optimization via indexing on key fields (store_id, product_id, etc.)
- Ensures:
  - Data consistency
  - Reduced redundancy
  - Faster queries
- Recommended cloud deployment (AWS RDS) for:
  - Scalability
  - Backup & recovery
  - High availability

Conclusion

- Developed a centralized, scalable database system for ABC Foodmart
- Combined RDBMS + ETL + analytics to improve operations
- Enabled insights into:
  - Sales performance
  - Inventory management
  - Profitability
- Value delivered:
  - Improved accuracy
  - Better decision-making
  - Scalable infrastructure for future growth